DailySeed · 产品设计文档

每天一粒种子。留下来的，会长成你的习惯。

版本 v1.4 · 面向个人开发者独立实现

v1.1 变更：改为 PWA 且**无推送** · **匿名 ID + 同步码** · **毕业由用户决定（不设阈值）** · **补留最近 3 天** · 产品名定为 **DailySeed**

v1.2 变更：新增**第三类卡「放纵」**与**放纵日**机制（§3.5）——卡片库扩到 100 张（含 20 张放纵卡）· 难度新增 0 档 · 门槛用「累计」而非「连续」，且**不公示**

v1.3 变更：放纵卡明确**两种取向**（停止型 / 给予型）——它不只是免罪符，也可以是「按你自己的预算吃一顿好的」（统计口径按实际卡片核算：HABIT 42 / EXPLORE 38）

v1.4 变更：新增**卡片「?」依据展开**（§3.6）——默认折叠、只陈述不劝服、依据分 A/B/C 三级，**没有依据的卡不显示入口**

写在最前

一句话

每天给你一个很小的约定。想做就做。觉得好就留下它——留久了，它会自己长成你的习惯。

起点：一个被所有人忽略的矛盾

所有习惯类产品都在问同一个问题：

```
「你今天做了吗？」
```

然后给你一个连续天数。断一天，归零。然后你就再也不打开了。

DailySeed 问的是另一个问题：

```
「这张，你还想要吗？」
```

系统不做监工，只做提醒者。

这是整个产品的原点，也是后面每一个设计决策的依据。

七个亮点

1｜每天一张随机的卡，你永远不知道会抽到什么

打开它，领取今天的一粒种子：

```
今天吃饭时，手机放在看不见的地方
```

明天可能是完全不同的事。**每天在「功课」「惊喜」和「许可」之间随机**——这个不确定性就是产品的味道。

卡片分三类：**习惯类**（可重复、会毕业）、**探索类**（一次性、制造惊喜）、**放纵类**（零消耗，什么都不用做）。

2｜没有打卡，没有连续天数，没有红点

唯一的动作是一个按钮：**「留着」**。

	打卡	**留着**
本质	向系统**汇报**	为自己**保留**
心理	"我得证明我做了"	"这张我想要"
断掉	归零 → 彻底放弃	无损失 → 继续

漏了就漏了。**Lally 等人的研究发现，偶尔漏掉一天不会破坏习惯的养成**——所以产品里根本没有 streak 这个东西。这不是审美选择，是有研究支持的。

> 这条研究还有第二层用处：**既然「漏一天」无害，它就可以被正大光明地做成产品的一部分**——见 §3.5「放纵日」。

3｜留下来的会反复回来，直到成为你的习惯

你留过的卡，会**更频繁地出现在你面前。**

```
反复遇到 + 反复做  =  习惯
```

这个过程**不靠记录，靠重复。**所以"我的花园"里显示的不是"连续 12 天"，而是：

```
我的花园：7 粒种子
  吃饭不看手机        留了 21 次
  睡前不刷屏          留了 14 次
  每天走 5000 步      留了 9 次
```

"连续"是脆弱的（断一次就崩），"收集"是稳固的（不会归零）。

4｜毕业由你宣布，系统绝不催促

```
系统：「『吃饭不看手机』你已经留了 21 次，还需要我提醒你吗？」
你：「不用了，它是我的一部分。」
   → 毕业。进入「已内化」，我不会再提醒你。
```

系统**不设任何阈值判定**，也不会主动催你。它只是把入口放在那里，什么时候用，你说了算。

系统主动放手的那一刻，是产品最想给你的东西。

5｜我们不让你「每天跑 5 公里」——这是有理由的

「21 天养成一个习惯」是个流传了 60 年的误解——它来自 1960 年一位整形外科医生的**临床观察**，不是实验。

真实研究（**Lally et al. 2010，伦敦大学学院**）说：

中位数	**66 天**（不是 21 天）
范围	**18 ~ 254 天**
决定快慢的是	**行为复杂度**

简单动作（"午饭时喝一杯水"）约 20 天就自动化了；复杂行为（"做 50 个仰卧起坐"）需要长得多。

所以我们只给你很小的动作。 100 张卡里，**51 张是难度 1 的轻动作，20 张是零消耗的放纵卡**，最高难度只有 3 张（3%）。

如果难度高的卡变多，这个产品就退化成了**又一个让人挫败的自律 App**。

6｜有些日子里，它会主动允许你不做，或者让你对自己好一点

所有自律产品都在做同一件事：**加约束、加难度、加惩罚。**

DailySeed 在池子里放了第三类卡：**放纵卡。**

```
今天睡到自然醒，不设闹钟
今天可以说一次「我不想」，不解释理由
今天按你自己的预算，吃一顿好的
今天买一样没用但你很喜欢的东西，按你自己的预算
```

抽到它，你可以什么都不做，也可以去做那件一直想做的事——**两种都不算破功。**

别的产品发任务，我们发许可。

放纵卡有两种取向：

- **停止型**：那件你不想做、却一直在硬撑的事 —— 一张免罪符

- **给予型**：那件你其实想做、却总觉得「不值当」的事 —— 一次主动的善待

共同点是**今天不用有理由。**

完整机制见 §3.5。

7｜卡片上有一个「?」，点开是它的来由

每张卡都能展开一句「为什么会有人想到这件事」。

有研究就写研究，只有机制就写机制，**什么都没有的卡就不显示这个入口。**

> 别的产品告诉你「要坚持」。
>
> 我们告诉你「这件事是谁发现的」。
>
> 详见 §3.6。

一句总结

> 别的产品帮你**管理任务**。
>
> DailySeed 帮你**收集习惯**。
>
> **你不是在坚持，你是在收集。**
>
> **而收集的东西，会变成你自己的。**
>
> **而且它允许你有不做的那一天，也提醒你对自己好一点。**

它不是什么

❌ 不是	因为
打卡 App	没有打卡，没有连续天数
待办清单	不列"你还没做的"
冥想/正念 App	我们给的是生活里的动作，不是音频内容
又一个自律工具	我们不要求你坚持任何事；还会主动给你一张放纵卡
社交产品	没有好友、没有排行榜、没有分享压力（一期）
消费主义推手	花钱的卡从不写金额，边界由你自己的预算定

一、一页纸概览

项	内容
产品名	DailySeed（中文：每日一粒）
一句话	每天一粒种子。觉得好就留下它，留久了它就成了你的习惯。
形态	**PWA**（移动端优先，**无推送**，用户主动打开）
登录	**匿名 ID + 8 位同步码**（零注册，无密码）
目标用户	想养成好习惯、但受不了打卡压力的年轻人（学生、刚工作）
核心循环	抽卡 → 去做 → 留着 → 反复出现 → 用户宣布毕业

卡片类型	习惯 HABIT / 探索 EXPLORE / **放纵 INDULGE**（§3.5）
放纵日	累计留住 ≥5 张后可主动把今天换成一张放纵卡；门槛**不公示**，冷却 7 天
依据	每张卡可展开「?」，写清研究依据 / 机制 / 无依据（§3.6）
与竞品的根本区别	打卡类产品问「你做了吗」；本产品只问「这张还要吗」
后端	Spring Boot 3 + Java 21 + Postgres
MVP 周期	2-3 周（每天 3-4 小时）

二、产品定位

2.1 核心洞察

打卡之所以失败，不是因为人懒，是因为打卡本身是一种"汇报"。

	打卡	**留着**
本质	向系统**汇报**	为自己**保留**
用户心理	"我得证明我做了"	"这张我想要"
断掉的后果	连续天数归零 → 彻底放弃	无损失 → 继续
系统的角色	监工	提醒者

> **产品第一原则：系统不做监工，只做提醒者。**
>
> 它不问你"做了没有"，只问"这张还要吗"。

2.2 随机与累积如何共存

```
输入层（随机性）    每天一张不同的卡              ← 保留
输出层（累积性）    哪些卡被留下来了、内化了      ← 习惯的成果
中间层（打卡）      不存在                        ← 弱化掉
```

随机和累积**不冲突**，因为它们在**不同的层**。

- ❌ 错误：累积放在输入层 → "连续 30 天做同一件事" → streak 地狱
- ✅ 正确：累积放在输出层 → "我收集了 7 个习惯" → 不会归零的成就

2.3 竞争定位

类别	**代表**	**差异**

打卡类	滴答清单、小习惯、Streaks、Loop	它们要求连续；我们不要求
习惯养成类	Habitica、21 天计划	它们固定同一件事；我们每天随机
冥想/正念类	潮汐、Now	它们内容固定；我们是生活动作
内容社区类	小红书自律帖	我们是工具体验，不是内容消费

本产品没有直接竞品，因为它没有类别。 这是它最重要的资产——人们只能问"你怎么想到的"。

而且几乎没有哪个自律产品敢主动递给你一张「今天可以不做」的卡。

2.4 「21 天」这个数字怎么用

重要：产品逻辑里不使用任何阈值。 但「关于」页面要讲这件事——它是品牌态度，也是传播点。

而且它不只是科普——Lally 研究的三条发现，恰好是产品的三个设计依据。

研究事实

「21 天养成一个习惯」不是研究结论。

它来自 1960 年整形外科医生 Maxwell Maltz 的临床观察（病人大约需要 21 天适应新面孔）。那是一次观察，不是实验，但被当成科学结论传了 60 年。

真实研究是 **Lally et al. (2010)，伦敦大学学院**：

《How are habits formed: Modelling habit formation in the real world》

European Journal of Social Psychology, 40(6), 998–1009

- 追踪 96 人 12 周
- 达到"自动化"的**中位数：66 天**
- **范围：18 到 254 天**

三条发现 ↔ 三个设计决策

#	研究发现	对应的产品决策
①	中位数 66 天，不是 21 天	不宣称任何天数，**毕业由用户决定**
②	**范围 18-254 天，快慢取决于行为复杂度**：简单行为（如"午饭时喝一杯水"）约 20 天自动化；复杂行为（如"做 50 个仰卧起坐"）需要长得多	**只给很小的动作**——不让你"每天跑 5 公里"，因为你被刻意放在范围里最快的一端
③	**偶尔漏掉一天，不会破坏习惯的养成**	**不搞连续天数、不打红点、不设 streak**；漏了就漏了，明天继续

第 ② 条是产品的立身之本：约束必须小。这不是审美偏好，是有研究支持的决策。

第 ③ 条让"不搞连续天数"从"我喜欢这样"变成"研究支持这样"。

第 ③ 条还有一次正面用法：如果漏掉一天无害，那「允许你不做」就不需要道歉——于是它变成了 §3.5 的**放纵日**。别人用它论证「不必自责」，我们把它做成了产品的一部分。

「关于」页面文案

```
关于「21 天」

你可能听过「21 天养成一个习惯」。

这个说法来自 1960 年一位整形外科医生 Maxwell Maltz 的临床
观察——他发现病人大约需要 21 天适应新面孔。
那是一次观察，不是实验。但它被当成科学结论传了 60 年。

真实的研究是这样的：

Lally 等人在伦敦大学学院做了一项实验，追踪 96 个人 12 周，
观察他们养成新习惯的过程。论文发表在《欧洲社会心理学杂志》。

他们发现了三件事，而这三件事正好是 DailySeed 的设计依据：

一、达到「自动」的中位数是 66 天，不是 21 天。
    但这个数字因行为而异，范围从 18 天到 254 天。

二、快慢取决于行为的复杂度。
    「午饭时喝一杯水」这类简单动作，二十天左右就自动化了；
    「做 50 个仰卧起坐」这类复杂行为需要长得多。
  → 所以我们只给你一个很小的动作。
    我们不让你「每天跑 5 公里」，因为研究说那需要几个月。

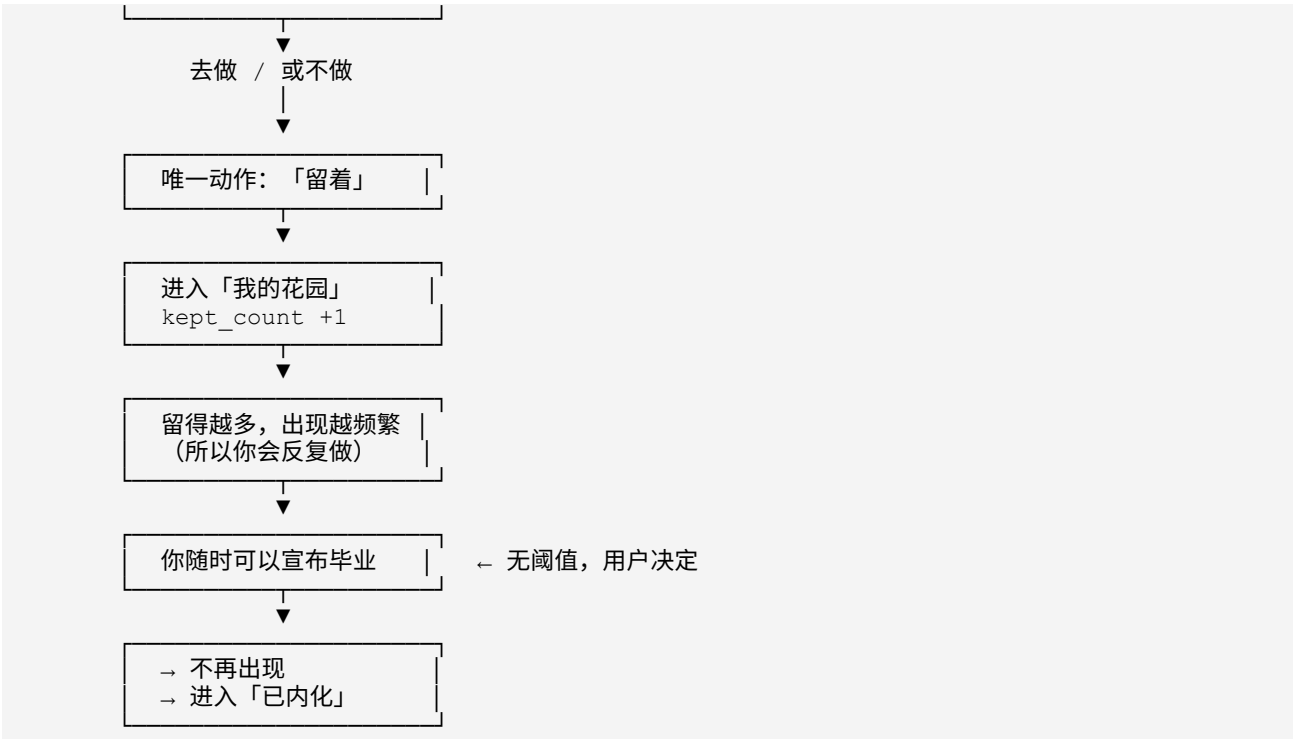
三、偶尔漏掉一天，不会破坏习惯的养成。
  → 所以我们没有连续天数，没有打卡，没有红点。
    漏了就漏了，明天继续。
```

最后那两句"所以我们……"，是整段文案的价值所在——它把产品设计从"我喜欢这样"变成了"研究支持这样"。

三、核心机制

3.1 完整生命周期

```
┌──────────────────────┐
│  用户主动打开 PWA     │   ← 没有推送，我们自己不叫他
└──────────┬───────────┘
           ▼
┌──────────────────────┐
│  领取今天的一粒种子   │   ← 懒生成，幂等
```



3.2 卡片分三类（关键设计）

类型	例子	能内化吗	承担
习惯类 HABIT	吃饭把手机放远 / 睡前 30 分钟不刷屏 / 每天走 5000 步	✅ 可重复、可毕业	**养成**
探索类 EXPLORE	走进第一家店 / 数路过的树 / 拐一个没走过的路口	❌ 一次就完了	**惊喜 + 分享**
放纵类 INDULGE	睡到自然醒 / 说一次「我不想」 / 买一样没用但喜欢的东西	❌ 不必做，也不留	**允许 + 惊喜**

三类混着发，你每天不知道会抽到「功课」「惊喜」，还是一张许可——这个不确定性就是产品的味道。

- 习惯类：进入「我的花园」，可留、可毕业
- 探索类：进入「探索记录」，一次性，但计入总探索数
- **放纵类：什么都不进**——不进花园、不计次数、不统计、不可毕业。它只是一天的天气

  旁路：放纵日——满足条件的那一天，你可以不领种子，改成点「今天想放纵」抽一张放纵卡（见 §3.5）。

3.3 「留着」的设计细节

「留着」是产品唯一的核心交互，必须**零压力**：

设计点	做法
不做强提醒	不弹"今天完成了吗"
允许忽略	什么都不点 = 默认丢弃，明天来新的
允许补留	**过去 3 天内的卡还能留**（见下）
不做否定反馈	不留 = 静默消失，不说"你放弃了"
一句话文案	按钮写「留着」，不写「完成」或「打卡」
放纵卡不适用	放纵卡没有「留着」按钮——它不需要被保留，也不需要被复现

补留机制（最近 3 天）

补留 = 允许你回头，把过去没留的卡重新留下。

	没有补留	有补留
昨天的卡	过期 → 永久丢弃	还能在历史里点「留着」
用户感受	"我错过了"	"我随时可以回头"

- **窗口：3 天**（含今天）
- **为什么是 3 天**：既能救急（忘了点、或今天才真正做了），又**避免任务堆砌**——超过 3 天就让它过去，保持"每日"的节奏
- 补留的卡同样 `kept_count +1`

3.4 进度呈现

❌ **不做：**

```
连续打卡 12 天            ← 断一次就崩，且逻辑上不成立
```

✅ **要做：**

```
我的花园：7 粒种子

┌──────────────────────────┐
│ 吃饭不看手机        留了 21 次  │
│ 睡前不刷屏          留了 14 次  │
│ 每天走 5000 步       留了 9 次  │
│ 对家人说句关心的话    留了 6 次  │
└──────────────────────────┘

本月新种下 2 粒 · 探索了 18 个生活实验
```

"连续"是脆弱的，"收集"是稳固的。 注意措辞：用「粒」「种下」延续种子隐喻。

放纵卡不进花园，也不统计。 不显示「本月放纵 3 次」——一旦记账，放纵就变成了业绩，又绕回了打卡。

3.5 放纵日（第三类卡）

为什么要有

所有自律产品都在做同一件事：**加约束、加难度、加惩罚。**

而 Lally 研究的第 ③ 条说：**偶尔漏掉一天，不会破坏习惯的养成**（§2.4）。

既然漏一天无害，它就可以被正大光明地做成产品的一部分——而不是一个需要被原谅的意外。

放纵日就是这句话的产品化。它同时承担两件事：

作用	说明
降低压力	用户知道自己随时能停，反而更愿意开始
增加惊喜	每天不知道会抽到「功课」「惊喜」还是「许可」——不确定性更强了

它不是什么（重要）

❌ 不是	因为
奖励	奖励意味着平时是苦的。**放纵不是挣来的，是天气**
积分 / 徽章	一旦可积累，用户就会为了攒它而表演自律
惩罚的反面	我们没有惩罚，所以也不需要赦免机制
一个不做的借口	它只占抽卡权重的 **10%**，且不可连抽

别的产品发任务，我们发许可。

而许可有两种：**免罪符**（那件你不想做、却一直在硬撑的事）和**善待**（那件你其实想做、却总觉得不值当的事）。

两者都叫放纵，都不需要理由。

两种取向（停止型 / 给予型）

取向	用户心里那句话	卡片在做什么	例子
停止型（免罪符）	「我今天真的不想做」	解除内疚	睡到自然醒 / 今天不列任何计

			划 / 可以说一次「我不想」
给予型（善待）	「我想要那个，但总觉得不值当」	主动给予	按自己的预算吃一顿好的 / 买一样没用但喜欢的东西 / 把一部剧看完

比例建议接近 1:1。 如果放纵卡全是「停止型」，产品会被读成躺平；

而**真正会被截图分享的是给予型**——「这个 App 让我去吃顿好的」比「允许我什么都不做」更有传播力。

涉及花钱的卡一律不写金额。 写「按你自己的预算」，不写「30 块以内」：

① 系统不该规定你花多少；② 自己定的数天然是事后不后悔的数；③ 产品不会被读成在推消费。

触发方式：主动入口，不公示门槛

今天抽到卡之后，「今天」页上有一个**低调入口**：

```
今天想放纵
```

点它 → 今天那张作废 → 换成一张 `INDULGE` 卡。

三个条件必须同时满足才出现（**全部不公示**——没有进度条、没有「还差几天」、没有「你已自律 N 天」）：

条件	阈值	为什么
累计留住过	≥ 5 张	用**累计**而不是**连续**——累计不会归零（§3.4）
冷却	距上次放纵 ≥ 7 天	防止变成每周固定的「放纵日仪式」
不连抽	昨天不是放纵卡	放纵不能成为一种连续状态

★ 门槛存在，但用户永远不知道还差多少。

一旦公示，放纵就变成一个目标，用户会为了攒它而表演自律——那正是我们要避免的打卡逻辑。

代价是：可能有人一直没发现这个入口。这是**故意接受的代价**，也是阶段 1 要观察的指标。

抽到放纵卡之后

项	行为
要不要做	停止型：什么都不用做；给予型：去做那件小事。**两种都不用汇报**
能「留着」吗	❌ 不能。留着意味着反复出现 = 反复放纵，语义崩坏
进花园吗	❌ 不进。花园里只有你收集的习惯
计入统计吗	❌ 不计。**不显示「本月放纵 3 次」**——一旦记账，放纵就成

	了业绩
能毕业吗	❌ 不能（§6.1）
能分享吗	✅ 推荐（`shareable = TRUE`）——这是全库最容易传播的一张

内容红线

唯一判据：做完之后不后悔。

✅ 可以写	❌ 绝不写
睡到自然醒，不设闹钟	熬夜到两三点
点一份平时不敢点的外卖，不看热量	吃到撑得难受
按你自己的预算，吃一顿你真正想吃的	为了"划算"点一堆吃不完的
买一样没用但喜欢的东西（按你自己的预算）	超出自己预算的消费（借钱、分期、刷爆）
看完一整部剧，中间不干别的	通宵刷手机
今天说一次「我不想」，不解释	对人发一次脾气
今天不列任何计划	今天什么都不管（欠账型）

> 放纵卡卖的是**停止**、**允许**和**给予**，不是**失控**。写偏一张，产品就从「温柔」变成「不负责任」——而且会被截图嘲笑。

与第一原则是否冲突？

不冲突，但必须靠三个设计挡住滑坡：

1. **门槛用累计，不用连续** —— 系统不需要知道你「这几天自律了」，只需要知道你「收集过 5 粒」（§3.4）
2. **门槛不公示** —— 不构成胡萝卜，用户无法为了它而努力
3. **放纵不进花园、不计账** —— 不是成就，无法炫耀，也就无法被追逐

> 换句话说：**它认可的不是「你坚持了」，而是「你已经在收集了」。**

示例卡（完整 20 张见《DailySeed-卡片库 v2》）

#	卡片	取向	难度
81	今天睡到自然醒，不设闹钟	停止型	0
82	今天按你自己的预算，吃一顿你真正想吃的	给予型	0
83	今天把一部剧看完，中间不干	给予型	0

	别的		
84	今天允许自己什么都不做 30 分钟，不为此道歉	停止型	0
85	今天买一样没用但你很喜欢的东西，按你自己的预算	给予型	0
86	今天可以说一次「我不想」，不解释理由	停止型	0
87	今天点一份你平时不敢点的外卖，不看热量	给予型	0
88	今天拒绝一个你本来会答应的请求	停止型	0

3.6 卡片上的「?」：为什么会有这张卡

每张卡上有一个**很小的「?」**。点它，才展开：

```
为什么会有人想到这件事


① 手机只要在视线里，就会占掉一部分注意力
② 依据：Ward 等人 2017 年的实验发现，手机放在桌上（即使关机）也会降低可用的认知容量
③ 出处：Journal of the Association for Consumer Research
```

默认永远不展开，不点就看不见。 没有红点、没有角标、没有「了解详情」的催促。

它解决的是哪个问题

卡片是随机的，所以第一反应常常是「凭什么」。

```
没有「?」：今天上厕所不带手机          → 像一句随机的命令
有「?」：今天上厕所不带手机 (?)        → 一个小小的实验
```

「?」把"随机"从「儿戏」变成「惊喜」。 它是卡片质量的外部证明，也是最容易被截图的部分。

语气铁律（重要）

它**不能变成说服**。一旦变成说服，产品就从「提醒者」退回「监工」。

✅ 这样写	❌ 绝不这样写
「有人研究过这件事」	「你应该这样做」
「一个实验发现……」	「不这样做会……」
「这条其实没有研究依据」	「研究表明」（其实没有研究时）

三条硬规则：

4. **只陈述，不劝服**——不出现「应该 / 必须 / 否则」
5. **不承诺效果和时间**——呼应「不宣称任何天数」（§2.4）
6. **没有依据就承认**——有研究写研究，只有机制写机制，**两者都没有的卡不显示「?」**

第 3 条是信任资产。 一个敢说「这张我也说不出根据」的产品，比一个每张都写"研究表明"的产品可信得多。

而 100 张卡里**大部分确实没有专门研究**——越小的动作，越没人单独研究过。所以依据库里会有三种层级。

依据的三个层级

层级	含义	展示方式
A · 研究	有具体研究支持，能给出作者/年份/期刊	正常显示，附出处
B · 机制	没有专门研究，但有可解释的机制	显示，并写明「这是机制，不是研究结论」
C · 无依据	说不清为什么有效	**不显示「?」**——宁可少一个入口，也不编一个理由

依据库见《DailySeed-卡片库 v2》附：依据库。

两条内容红线

7. **有争议的研究一律不用。** 例如「意志力像肌肉一样会被耗尽」（自我损耗）在多项多实验室复制中未能重现——这类结论流传再广也不进依据库。
8. **引文必须能核实。** 作者、年份、期刊缺一不可；宁可降级成 B（机制），也不写一句没有出处的"研究表明"。

与第一原则的关系

可发现，但不催促；能解释，但不劝服。 入口放在那里，什么时候点，你说了算（同 §6.1 毕业入口的处理方式）。

四、卡片系统

4.1 参数组合模型（解决内容枯竭）

不要把卡片当"内容"，要把它当"参数组合"。

```
卡片 = 场合 × 动作 × 限制

场合 occasion：通勤 / 吃饭 / 独处 / 睡前 / 和人相处 / 走路 / 工作学习
动作 action：  换一种 / 反过来 / 慢一点 / 不做 / 只做一次 / 多做一件 / 观察
限制 constraint：你平时会选的那个 / 你最习惯的 / 你从没试过的 / 最便宜的那个 / 你最想避免的
```

场合	动作	限制	生成
走路	换一种	你平时会选的那个	今天回家，不许走你平时那条路
吃饭	只做一次	你从没试过的	今天点一样你从没点过的东西
和人相处	多做一件	最便宜的那个	今天对第一个帮你的人，把"谢谢"说完整
睡前	不做	你最习惯的	今天睡前 30 分钟不碰手机

7 × 7 × 5 = 245 种组合。

放纵卡用另一套参数模型——它不生成「动作」，它生成「许可」：

```
放纵卡 = 场合 × 许可动作 × 免责边界

许可动作 permit 分两族：
  停止族：不做 / 停下来 / 只做一次（不管标准） / 差不多就行
  给予族：选你最想要的那个 / 贵一点的 / 慢一点的 / 多花一点的
免责边界 excuse：不算破功 / 不用解释 / 明天不补 / 不用等值得 / 不用先挣到

★ 涉及花钱的卡一律不写金额，只写「按你自己的预算」。
```

场合	许可动作	免责边界	生成
睡眠	不做（停止族）	不算破功	今天睡到自然醒，不设闹钟
社交	只做一次（停止族）	不用解释	今天可以说一次「我不想」
吃饭	选你最想要的（给予族）	不用等值得	今天按你自己的预算，吃一顿你真正想吃的
空间	多花一点的（给予族）	明天不补	今天买一样没用但你很喜欢的东西，按你自己的预算

放纵卡的生成空间是 **8 × 8 × 5 = 320 种**，比 HABIT 的 245 还大。但它有一个额外特性：

每张放纵卡只能撑一次——第二次出现就贬值了（因为许可不能重复发放，见 §3.5）。

所以放纵卡的**数量必须给足**（20 张），不能像习惯卡那样靠高频复现。

4.2 分类体系（8 类）

类别	覆盖	默认类型
饮食	吃什么、怎么吃、和谁吃	混合
睡眠	睡前、起床、作息	HABIT
运动	走路、拉伸、爬楼	HABIT
注意力	手机、屏幕、专注	HABIT
社交	家人、朋友、陌生人	混合
观察	看、听、注意	EXPLORE
空间	收拾、整理、环境	混合
情绪	记录、感受、表达	混合

放纵卡不新增类别。 「放纵」是一个**类型**（`type`），不是类别（`category`）——它复用上面 8 类里的场合（睡到自然醒 → 睡眠；买没用但喜欢的东西 → 空间/情绪）。类型决定「这张卡怎么被使用」，类别决定「它发生在生活的哪一块」。

4.3 卡片字段

字段	说明
`text`	卡片正文（一句话，≤ 25 字）
`type`	HABIT / EXPLORE / **INDULGE**
`category`	8 类之一
`occasion` / `action` / `constraint_key`	参数组合来源
`difficulty`	**0（放纵，零消耗）** / 1（很轻）/ 2（中等）/ 3（有挑战）
`photo_hint`	是否适合拍照留念
`shareable`	是否适合生成分享卡
`why`	**为什么这张卡有效**（内部字段，质量守门）
`evidence`	展示给用户的依据（结论 + 依据 + 出处）；**为空则不显示「?」**
`evidence_level`	A 研究 / B 机制 / C 无依据（C 不出入口）

`why` 写不出来的卡，一律删掉。 这是卡片质量的唯一守门机制。

五、每日分配算法

5.1 生成时机：懒生成

没有推送，所以也不需要定时批量任务。

```
用户点开 → 检查今天有没有卡 → 没有就现在生成
                            → 有就直接返回

第一次生成时落库（UNIQUE 约束兜底），之后永远返回同一张
```

好处：不需要 cron、不需要批处理、不浪费资源给不来的用户。

5.2 硬约束

```
List<Card> candidates = activeCards
    .filter(c -> !graduated(user, c))              // 已毕业的排除
    .filter(c -> !assignedWithin(user, c, 30))     // 30 天内发过的不发
    .filter(c -> !c.category().equals(yesterday.category()))  // 不连续同类
    .filter(c -> c.type() != yesterday.type())     // 尽量交替类型（候选不足时放宽）
    .filter(c -> !c.isIndulgence() || canIndulge(user))   // 放纵卡：门槛 + 冷却 + 不连抽
    .toList();
```

5.3 软权重

```
double weight(Card c) {
    double base = 1.0;

    // 留过的卡更容易再出现（核心机制）
    int kept = userCards.keptCount(user, c);
    base *= 1 + Math.log1p(kept) * 2.0;

    // 太久没出现的略微加权（避免冷落）
    long daysSince = daysSinceLastAssign(user, c);
    base *= 1 + Math.min(daysSince, 60) / 60.0 * 0.5;

    // 难度匹配：用户最近留得多 → 可以上难度
    base *= difficultyMatch(user.recentKeepRate(), c.difficulty());

    // 类型权重：HABIT 6 : EXPLORE 3 : INDULGE 1
    // 放纵卡不参与「留得越多越频繁」的累积逻辑——它靠稀缺维持惊喜
    base *= typeWeight(c.type());

    return base;
}
```

5.4 幂等与确定性（关键工程点）

同一个用户、同一天，必须永远得到同一张卡。

```
// 用 (user_id, date) 作为随机种子 —— 无需锁，天然幂等
long seed = Objects.hash(userId, localDate.toString());
Random rng = new Random(seed);
Card pick = weightedRandom(candidates, rng);
```

加上数据库唯一约束兜底：

```
UNIQUE (user_id, assigned_date)
```

这两个机制合起来，每日分配完全幂等且无锁。 这是整个设计里最优雅的一处：不用定时任务、不用分布式锁、重试安全。

> **唯一例外：放纵日的主动换卡**（见 §5.5）。换卡后 `swapped_at` 落库，所以它同样是幂等的——「同一用户同一天永远得到同一张卡」这条铁律依然成立，只是那张卡允许被换一次。

5.5 放纵日的判定与幂等

下面四个条件必须同时满足，缺一不可：

```
boolean canIndulge(User user, LocalDate today) {
    // ① 累计留住过 ≥5 张（不是连续——累计永不归零）
    long keptKinds = userCards.countKeptKinds(user.id());
    if (keptKinds < 5) return false;

    // ② 冷却：距上次放纵 ≥7 天
    LocalDate last = assignments.lastIndulgenceDate(user.id());   // 无记录 = 很久以前
    if (last != null && ChronoUnit.DAYS.between(last, today) < 7) return false;

    // ③ 不连抽：昨天不是放纵卡
    if (assignments.typeOf(user.id(), today.minusDays(1)) == Type.INDULGE) return false;

    // ④ 今天还没换过
    return !assignments.hasSwapped(user.id(), today);
}
```

SQL 层面只有两条只读查询，**不需要新表、不需要定时任务**：

```
-- 累计收集（门槛）
SELECT COUNT(*) FROM user_cards WHERE user_id = ? AND kept_count > 0;

-- 上次放纵（冷却）
SELECT MAX(a.assigned_date)
FROM daily_assignments a JOIN cards c ON c.id = a.card_id
WHERE a.user_id = ? AND c.type = 'INDULGE';
```

★ 幂等性怎么保证（唯一允许的例外）

原来那条铁律是：**同一用户同一天，永远得到同一张卡**（§5.4）。

放纵入口是**唯一允许的例外**，但它自己也必须是幂等的：

```
第一次 POST /api/today/indulge
  → 今天那张卡作废，写入一张新的 INDULGE 卡，swapped_at = now()
  → 返回放纵卡

第二次 POST /api/today/indulge
  → 发现 swapped_at 非空 → 直接返回当前那张（不再换）
```

- 「每天最多换一次」由 `daily_assignments.swapped_at` 保证，**不是靠前端禁用按钮**
- 抽卡种子逻辑不变：放纵卡同样用 `hash(userId, date + ":indulge")` 选，重试安全
- 作废的原卡不补发、不回流（它只是没被选中）

  代价：`swapped_at` 是 `daily_assignments` 上唯一一个「生成后还可变」的字段。

  值得——因为它买到了「用户有主动权」这件事。

六、毕业机制

6.1 不设阈值，用户决定

产品逻辑里不存在"该毕业了"的判定。 系统永远不说"你该毕业了"。

> **放纵卡永不毕业。** 「放纵」不是要被内化的东西，也不存在「该不该放纵」的判定。

但必须解决可发现性，否则毕业永不发生：

```
习惯库 → 每张卡点进去
       → 底部常驻一个低调按钮：「它已经是我的习惯了」

当 kept_count >= 7 时
       → 列表里这张卡旁边出现一个小标记
       → 不是弹窗、不是红点、不推送
       → 只是告诉你"这个选项现在有意义了"
```

原则：入口可见，但绝不催促。

6.2 毕业流程

```
用户点击「它已经是我的习惯了」
          ↓
系统：「『吃饭不看手机』以后我就不提醒你了。确定吗？」
        [ 确定 ]    [ 再留一阵 ]
          ↓
       确定 → 毕业
          ↓
     · 不再出现在抽卡池
     · 进入「已内化」清单
     · 生成毕业卡（可分享）
     · ★ 此时提示用户抄下同步码（见 §6.4）
```

6.3 毕业是产品的情绪高点

系统主动放手这个动作，本身就是产品价值观的体现。

毕业卡文案：

> 「『吃饭不看手机』正式成为你的习惯了。

从今天起，我不会再提醒你这件事。」

你留了它 34 次。

毕业卡是最好的传播物料——它同时传达了成就感 + 产品的克制。

6.4 ★ 同步码提示的最佳时机

用户第一次毕业时，**弹出同步码并请他抄下来**：

```
「你的花园已经有 1 粒种子。
如果换手机或清了缓存，这颗种子会丢。
把这个码抄下来，随时能找回你的花园：」


        K7M2 - P9QX


 [ 我已经抄好了 ]    [ 复制 ]
```

为什么是这个时候：

- 太早 → 他没资产，不在乎
- 太晚 → 已经丢了
- **刚毕业时 → 他第一次真正拥有东西**，会心甘情愿抄下来

这是整个产品里我个人最喜欢的一处设计。

七、账号与身份：匿名 ID + 同步码

7.1 为什么不注册

	匿名 ID	账号
用户操作	**打开即用，0 步**	手机号/邮箱/微信授权
换手机	❌ 数据丢	✅ 自动同步
开发量	小	大（验证码、密码、找回）
流失点	无	**注册是最大的流失点**

核心矛盾：这个产品的**核心资产就是"我的花园"**。丢一次，用户基本不会回来。

7.2 方案：匿名 ID + 8 位同步码

```
打开即用
  → 本地生成随机 device_id
  → 服务端创建 user 记录 + 生成 8 位同步码（如 K7M2-P9QX）
  → 同步码存在本地
```

```
换设备 / 清缓存
  → 输入同步码 → 全部数据恢复
  → 绑定新设备
```

不需要密码、不需要验证码、不需要注册。 就是"一个码对应一个人"。

同步码设计：

- 8 位，去掉易混淆字符（`0/O`、`1/I/L`）
- 分组显示：`K7M2-P9QX`
- 可重置（旧码立即失效）
- 服务端只存哈希，不存明文（安全习惯，即使 MVP 也值得）

八、召回机制（因为没有推送）

8.1 我们不推送，让用户自己设

在用户第一次领取种子后，提供「给自己设个提醒」：

```
「要不要给你自己设个提醒？每天几点想看一眼？」
        ↓
  [ 下载日历提醒 ]   ← 生成 .ics 文件
  [ 每天早上 8:00 ]
  [ 不用了 ]
```

妙处在于：

- **不是我们推送，是用户自己设的**——技术零成本，心理完全不同
- 用户自己设提醒这个动作，本身就是一种**承诺**
- 完全符合"系统不做监工"

8.2 其他召回抓手

抓手	说明
「我的花园」页面	有资产就会想回来看
毕业卡	有情感高点，会分享
探索记录	"我已经探索了 23 个生活实验"
放纵卡的不确定性	「今天会不会是放纵卡」本身就是打开理由——不违反「不推送」原则

决策：不绑定已有动作（habit stacking）。 用户明确表示这个方案感觉不行，放弃。

8.3 关于留存的诚实判断

一个关于习惯养成的产品，如果必须靠推送才有留存，那它本身就失败了。

所以：**接受较低的打开频率，靠每次打开时的体验取胜。** 如果阶段 1 数据显示打开率太低，问题在卡片质量和体验，不在推送。

九、数据模型

```
-- ============ 卡片库 ============
CREATE TABLE cards (
    id              BIGSERIAL PRIMARY KEY,
    type            VARCHAR(16)  NOT NULL,   -- HABIT / EXPLORE / INDULGE
    category        VARCHAR(32)  NOT NULL,
    text            VARCHAR(64)  NOT NULL,
    occasion        VARCHAR(32),
    action          VARCHAR(32),
    constraint_key  VARCHAR(32),
    difficulty      SMALLINT     NOT NULL DEFAULT 1,   -- 0=放纵（零消耗） 1/2/3
    photo_hint      BOOLEAN      NOT NULL DEFAULT FALSE,
    shareable       BOOLEAN      NOT NULL DEFAULT FALSE,
    why             TEXT,                    -- 内部：为什么有效
    evidence        TEXT,                    --          NULL = 不显示「?」
    evidence_level  CHAR(1),                 -- A 研究 / B 机制 / C 无依据
    status          VARCHAR(16)  NOT NULL DEFAULT 'ACTIVE',
    created_at      TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- ============ 用户（零注册） ============
CREATE TABLE users (
    id              BIGSERIAL PRIMARY KEY,
    sync_code_hash  VARCHAR(64)  NOT NULL UNIQUE,  -- 只存哈希
    timezone        VARCHAR(64)  NOT NULL DEFAULT 'Asia/Shanghai',
    remind_at       TIME,                          -- 用户自设的提醒时间（用于生成 ics）
    created_at      TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE user_devices (
    id              BIGSERIAL PRIMARY KEY,
    user_id         BIGINT       NOT NULL REFERENCES users(id),
    device_id       VARCHAR(64)  NOT NULL UNIQUE,  -- 本地生成的匿名 ID
    last_seen_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- ============ 每日发放（幂等核心） ============
CREATE TABLE daily_assignments (
    id              BIGSERIAL PRIMARY KEY,
    user_id         BIGINT       NOT NULL REFERENCES users(id),
    card_id         BIGINT       NOT NULL REFERENCES cards(id),
    assigned_date   DATE         NOT NULL,
    state           VARCHAR(16)  NOT NULL DEFAULT 'PENDING', -- PENDING/KEPT/DISCARDED
    kept_at         TIMESTAMPTZ,
    photo_url       TEXT,
    note            VARCHAR(200),
    created_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
```

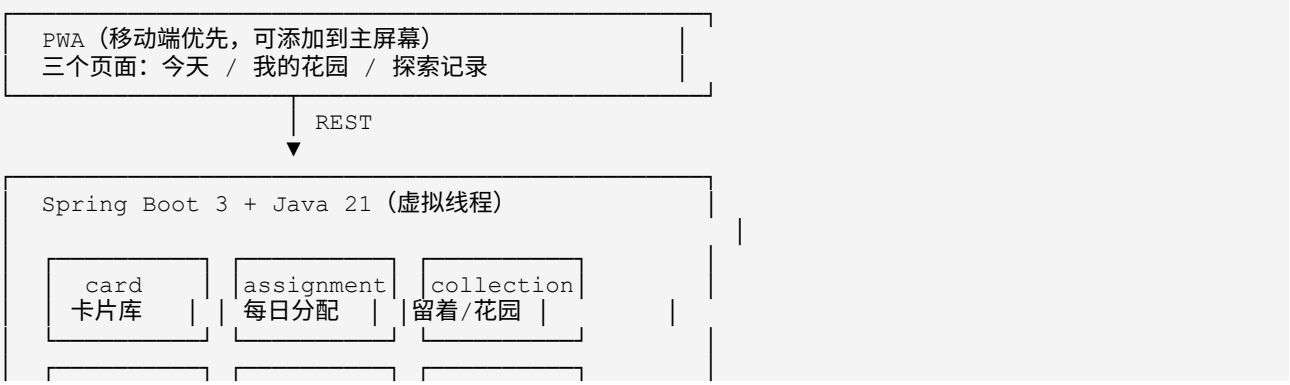
```
    swapped_at      TIMESTAMPTZ,                -- 非空 = 今天已用过「放纵」入口
    UNIQUE (user_id, assigned_date)             -- ★ 幂等保证
);
CREATE INDEX idx_assign_user_date ON daily_assignments(user_id, assigned_date DESC);

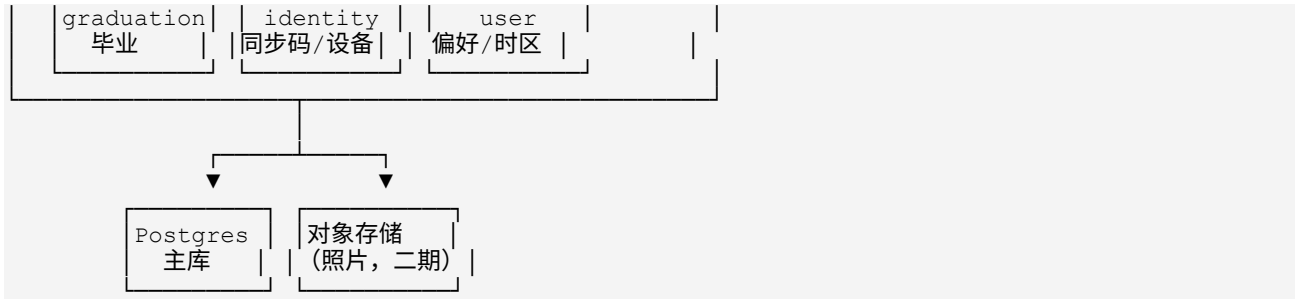

-- ============ 我的花园 ============
CREATE TABLE user_cards (
    id              BIGSERIAL PRIMARY KEY,
    user_id         BIGINT       NOT NULL REFERENCES users(id),
    card_id         BIGINT       NOT NULL REFERENCES cards(id),
    kept_count      INT          NOT NULL DEFAULT 0,    -- ★ 核心指标
    first_kept_at   TIMESTAMPTZ,
    last_kept_at    TIMESTAMPTZ,
    state           VARCHAR(16)  NOT NULL DEFAULT 'ACTIVE', -- ACTIVE/GRADUATED
    graduated_at    TIMESTAMPTZ,
    UNIQUE (user_id, card_id)
);
```

9.1 关键设计说明

设计	为什么
`daily_assignments` 的 `UNIQUE(user_id, assigned_date)`	幂等，防重复发放
`user_cards.kept_count`	**核心指标**，驱动抽卡权重和毕业入口显示
`sync_code_hash` 只存哈希	即使 MVP 也要有的安全习惯
`user_devices` 独立表	一个用户可绑多设备（换手机不换码）
`cards.why`	卡片质量守门
`cards.evidence` 为空则不显示「?」	宁缺勿编（§3.6）
`daily_assignments.swapped_at`	放纵日「每天最多换一次」的幂等保证（§5.5）
`cards.type` 含 `INDULGE`	第三类卡；不进花园、不统计
无任何推送相关字段	因为不推送

十、技术架构





没有 Redis，没有消息队列，没有定时任务。 一个人维护的系统，越少组件越好。

放纵日同样**不引入任何新组件**——它是分配算法里的一个分支，加一次 `COUNT` 和一次 `MAX(date)` 查询。

10.1 Java 21 特性的实际用处

特性	用在哪
虚拟线程	批量接口（如导出、统计），二期批量生成
Record	DTO、卡片参数组合、选卡决策结果
Pattern Matching	分配状态机、毕业状态流转

10.2 模块划分（Spring Modulith 模块化单体）

```
com.xxx.dailyseed
├── card          卡片库、参数组合、质量守门
├── assignment    每日分配算法、幂等保证、补留窗口
├── collection    留着、我的花园、探索记录
├── graduation    毕业入口、毕业仪式
├── identity      匿名 ID、同步码、设备绑定
└── user          用户、偏好、时区
```

不要上微服务。 单人项目，模块化单体是唯一正确选择。

十一、接口设计（MVP）

```
POST   /api/session
       → 首次进入：创建匿名 ID + 返回同步码
       body: { deviceId }


POST   /api/session/restore
       → 用同步码恢复数据
       body: { syncCode, deviceId }


GET    /api/today
       → 今天的一粒种子（懒生成，幂等）
       → 响应里直接带 `evidence`（体积很小，不为它单开接口）


POST   /api/today/keep
```

```
        → 留着今天的
        body: { photoUrl?, note? }


GET     /api/history?days=3
        → 最近 3 天可补留的卡


POST    /api/history/{assignmentId}/keep
        → 补留（仅限 3 天内）


GET     /api/garden
        → 我的花园（含 keptCount、毕业入口显示条件）


POST    /api/garden/{cardId}/graduate
        → 宣布毕业


GET     /api/explored
        → 探索记录


POST    /api/today/indulge
        → 把今天换成一张放纵卡（需满足门槛与冷却；每天最多一次）


GET     /api/reminder.ics
        → 生成日历提醒文件（用户自设提醒）
```

设计原则：接口极少。 核心只有三个动作——拿到今天那张、留着、毕业。

（外加一个可选的第四动作：**今天想放纵。**它不新增页面，只是「今天」页上的一个低调入口。）

十二、MVP 边界

第一版必须做（2-3 周）

#	功能	说明
1	卡片库 100 张	8 类别；HABIT 42 / EXPLORE 38 / **INDULGE 20**
2	匿名 ID + 同步码	零注册，可恢复
3	每日一卡（懒生成 + 幂等）	种子随机 + 唯一约束
4	留着 / 不留	唯一核心交互
5	补留（最近 3 天）	降低压力
6	我的花园	含已留次数、毕业入口
7	毕业（无阈值）	含毕业卡
8	`.ics` 日历提醒	召回抓手，零推送成本
9	极简 UI	三页：今天 / 花园 / 探索
10	**放纵日**	第三类卡 + 「今天想放纵」入口（门槛

		不公示）
11	卡片「?」依据展开	首批：放纵卡 20 张 + 阶段 0 用卡（§3.6）

第一版坚决不做

❌ 不做	为什么
任何形式的推送	已决定：用户主动打开
绑定已有动作（habit stacking）	用户明确否掉了
拍照留念	增加摩擦，二期再加
分享卡片	先验证核心循环
好友 / 社交	冷启动杀手
个性化推荐算法	数据量不够，规则就够
毕业阈值判定	已决定：用户决定
App（原生）	PWA 足够
Redis / MQ / 定时任务	组件越多，单人越难维护
放纵次数统计 / 进度显示 / 排行榜	一旦公示，放纵就变成目标（见 §3.5）

判断标准：任何不服务于「抽卡 → 留着 → 毕业」这个循环的功能，第一版都不做。

十三、验证路径与指标

阶段 0：手工验证（本周，0 行代码）★ 不能跳过

做法：把 100 张卡写出来，用微信群/备忘录，**每天手动给 3-5 个朋友发一张。**

验证目标：

- 卡片有没有魅力（有人主动说"这张有意思"吗）
- 有没有人愿意"留着"
- **种子/惊喜感成不成立**
- **放纵卡的反应是否强于习惯卡**（发的时候加一句：「今天这张不用做，也可以去做，反正不算破功」；**停止型和给予型各发一张**，看哪一种反应更强）
- **有人点开「?」吗**——点了，说明卡片引起了好奇；有人**转述**里面的研究，是最强信号

成本：一个下午。**技术做得再好，卡写得无聊，产品就是死的。**

阶段 1：最小可用（2-3 周）

做法：MVP 上线，拉 10-20 个真实用户。

指标	目标	说明
次日打开率	> 50%	**最重要**（无推送，纯靠体验）
留着率	> 40%	每天有多少人点「留着」
7 日留存	> 30%	无推送情况下到 30% 就是好成绩
补留使用率	观察	验证 3 天窗口是否合适
有人主动毕业	至少 1 人	验证毕业机制可被发现
放纵入口使用率	观察	太高 = 门槛失效；为 0 = 入口看不见（§3.5）

> **注意**：因为没有推送，打开率目标要下调预期。**如果次日打开率 < 30%，问题在卡片质量和体验，不在推送。**

阶段 2：传播（第 4-6 周）

- 加拍照留念
- 加毕业卡分享
- 扩到 100 用户
- 验证：有没有人主动分享？自然增长率？

十四、风险与未决问题

风险

#	风险	严重度	应对
1	**无推送 → 打开率低**	🔴 高	`.ics` 自设提醒 + 花园的资产感；阶段 0 先验证内容
2	卡片内容枯竭	🟡 中	参数组合（245 + 128）+ `why` 守门
3	**用户永远不毕业**	🟡 中	毕业入口可见但不催促；kept≥7 时给低调标记

4	"留着"太轻，用户懒得点	🟡 中	按钮文案 + 位置设计；可 A/B 测试
5	同步码丢失 = 用户流失	🟡 中	第一次毕业时强提示；可复制
6	定位模糊（工具？趣味？）	🟡 中	保持"种子/实验"调性，不叫"习惯养成"
7	三类卡比例失衡	🟢 低	目标 6:3:1（HABIT : EXPLORE : INDULGE），用权重控制
8	**放纵卡被当成「放纵许可」滥用**	🟡 中	门槛（累计 ≥5）+ 冷却（7 天）+ 不连抽 + 权重 10%；门槛不公示，避免「攒放纵」
9	放纵卡写偏 → 变成不负责的建议	🟡 中	**内容红线：做完之后不后悔**；禁止熬夜、暴食、超出自己预算的消费
10	**「?」被写成劝服 → 退回监工**	🟡 中	语气铁律（不出现应该/必须/否则）+ 不承诺效果；只陈述「谁发现了什么」
11	引用错误或已被否定的研究 → 信任崩塌	🔴 高	引文逐条核实（作者+年份+期刊）；有复制争议的一律不用；宁可降级成 B 机制

已决定（v1.4）

问题	决定
PWA 还是小程序	✅ **PWA**
推送	✅ **不做**，用户主动打开
每日发放	✅ **懒生成**（点开才生成）
账号体系	✅ **匿名 ID + 8 位同步码**
毕业阈值	✅ **不设，用户决定**
补留	✅ **允许，最近 3 天**
绑定已有动作	✅ **不做**（用户否掉）
产品名	✅ **DailySeed**（中文：每日一粒）
提醒方式	✅ **用户自己设（.ics）**，可选
放纵日	✅ **做**——第三类卡 INDULGE；门槛 = 累计留住 ≥5 张（**不公示**）+ 冷却 7 天；不可连抽；不进花园、不可毕业

卡片「?」	✅ **做**——默认折叠、不催促、不劝服；依据分 A/B/C 三级；**无依据则不显示入口**

真正未决

9. 花园页面的视觉隐喻（种子/植物/花园，怎么画）
10. 毕业卡的分享形式（图？文字？）
11. 卡片 `difficulty` 字段的分布控制（研究说简单行为更快自动化，所以难度 3 的卡应该占比很低）
12. 放纵日的门槛**是否应该公示**——当前决定不公示（避免用户为了攒放纵而表演自律），但需在阶段 1 验证用户会不会因此找不到入口或感到困惑
13. 依据库剩下 80 张的补全节奏；C 级卡是否改为显示「这张我也说不出根据」（当前：不显示入口）

十五、下一步

顺序	任务	产出
1	**补足卡片库到 100 张**（含 20 张放纵卡）	卡片数据
2	**阶段 0 手工验证**	真实反馈，决定要不要继续
3	搭 Spring Boot 骨架	可跑的 `/api/today`
4	实现分配算法	幂等 + 加权随机
5	实现留着 + 补留	核心循环闭环
6	实现花园 + 毕业	情感高点
7	PWA 前端 + `.ics` 提醒	可上线